BÀI THỰC HÀNH ĐỆ QUY – BACKTRACKING (PYTHON)

Kỹ thuật **Backtracking (Quay lui)** sử dụng đệ quy trong lập trình là một kỹ thuật nền tảng nhưng rất mạnh mẽ, giúp giải quyết một lớp các bài toán tìm kiếm và tối ưu.

Chúng ta sẽ đi từ những ví dụ cơ bản nhất để hiểu cách "thử" (try) và "quay lui" (backtrack) cho đến các bài toán phức tạp hơn.

Mục Đích & Yêu Cầu Chung

Mục Đích

- **Hiểu rõ bản chất:** Nắm vững khái niệm "không gian trạng thái" (state space) và cách đệ quy duyệt qua không gian này.
- **Thực hành cốt lõi:** Luyện tập việc xác định "trạng thái" (state), các "lựa chọn" (choices) tại mỗi trạng thái, và thời điểm cần "quay lui".
- **Xây dựng giải pháp:** Học cách xây dựng một hàm đệ quy backtracking hoàn chỉnh: điều kiện dừng (base case), vòng lặp thử các lựa chọn, và hành động "hoàn tác" (undo) sau khi đệ quy trả về.

Yêu Cầu Chung

- **Ngôn ngữ:** Tất cả bài tập đều phải được cài đặt bằng **Python 3** (khuyến khích Python 3.10+).
- **Kỹ thuật:** Bắt buộc sử dụng đệ quy và kỹ thuật backtracking. Không dùng vòng lặp (iterative) để khử đệ quy, trừ khi được yêu cầu.
- **Môi trường:** Viết dạng script Python (`.py`) hoặc chạy trên Jupyter Notebook / Google Colab / VS Code.
- **Độ rõ ràng:** Code tuân thủ chuẩn PEP 8, có chú thích (comment) rõ ràng giải thích các bước quan trọng (điều kiện dừng, thử, đệ quy, quay lui).

Bài 1: [Dễ] Liệt Kê Hoán Vị (Permutations)

Mô tả yêu cầu

Viết chương trình Python nhận vào một danh sách các số nguyên (ví dụ: `[1, 2, 3]`) và in ra tất cả các hoán vị (permutations) có thể có của tập hợp đó.

- **Input:** `[1, 2, 3]`
- **Output (thứ tự có thể khác):**

```
None
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
[3, 1, 2]
[3, 2, 1]
```

Gợi ý hướng dẫn

Chúng ta cần xây dựng hoán vị từng bước một. Giả sử hoán vị đang xây dựng được lưu trong một danh sách `current_permutation`. Chúng ta cũng cần một danh sách kiểu bool `used` để đánh dấu xem số nào đã được dùng.

Hàm đệ quy `backtrack(current_permutation, used)`:

1. **Điều kiện dừng:** Nếu `len(current_permutation) == len(nums)`, ta đã có một hoán vị hoàn chỉnh. In ra và `return`.
2. **Vòng lặp:** Duyệt qua tất cả các chỉ số `i` trong danh sách `nums`.
    - Nếu số `nums[i]` chưa được dùng (`used[i] == False`):
        - **Thử (Try):** Thêm `nums[i]` vào `current_permutation` và đánh dấu `used[i] = True`.
        - **Đệ quy:** Gọi lại `generate(...)` để đi sâu hơn.
        - **Quay lui (Backtrack):** Xóa số vừa thêm khỏi `current_permutation` (dùng `pop()`) và đặt `used[i] = False` (trả lại trạng thái cũ).

Code mẫu (Python)

```python
class Permutations:
    def find_permutations(self, nums: list[int]) -> None:
        current_permutation = []
        used = [False] * len(nums)
```

```
        self._generate(nums, current_permutation, used)

    def _generate(self, nums: list[int], current_permutation: list[int], used:
list[bool]) -> None:
        # 1. Điều kiện dừng: Khi đã xây dựng đủ 1 hoán vị
        if len(current_permutation) == len(nums):
            print(current_permutation)
            return

        # 2. Vòng lặp các lựa chọn
        for i in range(len(nums)):
            if not used[i]:
                # 3. Thử (Try)
                current_permutation.append(nums[i])
                used[i] = True

                # 4. Đệ quy
                self._generate(nums, current_permutation, used)

                # 5. Quay lui (Backtrack)
                used[i] = False
                current_permutation.pop()

# --- Chạy thử ---
if __name__ == "__main__":
    solver = Permutations()
    solver.find_permutations([1, 2, 3])
```

Vận dụng AI

- **Hỏi AI (Gemini, ChatGPT):** *"Hãy vẽ cây đệ quy (recursion tree) cho thuật toán tìm hoán vị với input là [1, 2] trong Python."*
- **Sử dụng AI gợi ý code:** Khi viết đoạn `current_permutation.pop()`, hãy tự hỏi: *"Nếu không có câu lệnh `pop()` và `used[i] = False`, kết quả của chương trình sẽ bị sai như thế nào?"*

Bài 2: [Dễ/Trung bình] Tìm Đường Đi Trong Mê Cung (Maze Solver)

Mô tả yêu cầu

Cho một mê cung 2D (ma trận danh sách lồng nhau `list[list[int]]`) trong đó `1` là đường đi (walkable) và `0` là tường (wall). Viết hàm tìm một đường đi từ điểm bắt đầu `(0, 0)` đến điểm kết thúc `(N-1, M-1)`.

Nếu tìm thấy, trả về `True` (và đánh dấu đường đi đó bằng cách đổi giá trị `1` thành `2`). Nếu không tìm thấy, trả về `False`.

Chỉ được phép di chuyển 4 hướng: **Lên, Xuống, Trái, Phải**.

Gợi ý hướng dẫn

Đây là bài toán backtracking cổ điển. "Trạng thái" chính là tọa độ `(x, y)` hiện tại.

Hàm đệ quy `solve(x, y)`:

1. **Điều kiện dừng (Thất bại):**
   - Nếu `(x, y)` ra ngoài biên mê cung.
   - Nếu `maze[x][y] == 0` (đụng tường).
   - Nếu `maze[x][y] == 2` (đã đi qua rồi, tránh lặp vô tận).
2. **Thử (Try):** Đánh dấu ô hiện tại là đã đi qua: `maze[x][y] = 2`.
3. **Điều kiện dừng (Thành công):** Nếu `(x, y)` là điểm đích `(N-1, M-1)`, trả về `True`.
4. **Đệ quy (4 hướng):**
   - Thử đi Xuống `(x+1, y)`, Lên `(x-1, y)`, Phải `(x, y+1)`, Trái `(x, y-1)`. Nếu bất kỳ hướng nào trả về `True`, ngay lập tức trả về `True`.
5. **Quay lui (Backtrack):**
   - Nếu cả 4 hướng đều `False` (ngõ cụt), ta phải hoàn tác: Đặt `maze[x][y] = 1` (trả lại trạng thái đường đi chưa thăm) và trả về `False`.

Code mẫu (Python)

Python

```python
class MazeSolver:
    def find_path(self, maze: list[list[int]]) -> bool:
        if not maze or not maze[0]:
            return False

        n = len(maze)
        m = len(maze[0])
```

```
        def solve(x: int, y: int) -> bool:
            # 1. Điều kiện dừng (Thất bại / Không hợp lệ)
            if x < 0 or x >= n or y < 0 or y >= m or maze[x][y] == 0 or maze[x][y]
== 2:
                return False

            # 2. Thử: Đánh dấu là thuộc đường đi / đã thăm
            maze[x][y] = 2

            # 3. Điều kiện dừng (Thành công)
            if x == n - 1 and y == m - 1:
                return True

            # 4. Đệ quy 4 hướng
            if solve(x + 1, y): return True  # Xuống
            if solve(x - 1, y): return True  # Lên
            if solve(x, y + 1): return True  # Phải
            if solve(x, y - 1): return True  # Trái

            # 5. Quay lui (Backtrack): Ngõ cụt, trả lại trạng thái cũ
            maze[x][y] = 1
            return False

        return solve(0, 0)

# --- Chạy thử ---
if __name__ == "__main__":
    grid = [
        [1, 0, 0, 0],
        [1, 1, 0, 1],
        [0, 1, 0, 0],
        [1, 1, 1, 1]
    ]

    solver = MazeSolver()
    if solver.find_path(grid):
        print("Tìm thấy đường đi!")
        for row in grid:
            print(row)
    else:
        print("Không tìm thấy đường đi.")
```

Vận dụng AI

- **Hỏi AI (Gemini):** *"Trong code tìm đường mê cung bằng Python, tại sao bước `maze[x][y] = 1` (quay lui) lại quan trọng? Điều gì xảy ra nếu tôi quên bước đó?"*
- **Tạo Test Case:** *"Hãy tạo giúp tôi một ma trận mê cung 10x10 trong Python biểu diễn dạng list of lists không có đường đi từ (0,0) tới (9,9)."*

Bài 3: [Trung bình] Bài Toán N Hậu (N-Queens)

Mô tả yêu cầu

Viết chương trình Python giải bài toán N-Queens: Đặt N con hậu lên bàn cờ $N \times N$ sao cho không có hai con hậu nào "ăn" nhau (không cùng hàng, cùng cột, hoặc cùng đường chéo).

Chương trình cần in ra tổng số lượng các cách đặt hợp lệ.

Gợi ý hướng dẫn

Vì mỗi hàng chỉ có thể đặt 1 con hậu, ta sẽ đặt hậu lần lượt từ hàng `0` đến hàng `N-1`.

Hàm đệ quy `place_queen(row)`:

- `positions[r] = c` có nghĩa là ở hàng `r`, con hậu được đặt ở cột `c`.

1. **Điều kiện dừng:** Nếu `row == N`, ta đã đặt thành công N con hậu. Tăng biến đếm số lời giải lên 1 và `return`.
2. **Vòng lặp:** Duyệt qua các cột `col` từ `0` đến `N-1`.
3. **Kiểm tra an toàn (`is_safe`):** Viết hàm kiểm tra xem việc đặt hậu tại `(row, col)` có bị đe dọa bởi các con hậu ở các hàng trước (`0` đến `row-1`) không.
    - Chỉ cần kiểm tra cột và 2 đường chéo (không cần kiểm tra hàng vì mỗi hàng ta chỉ chọn 1 ô).
4. **Thử (Try) & Đệ quy:** Nếu `is_safe` trả về `True`, gán `positions[row] = col` và gọi đệ quy `place_queen(row + 1)`.
5. **Quay lui (Backtrack):** Ở bài này, giá trị `positions[row]` sẽ tự động bị ghi đè ở vòng lặp `col` tiếp theo, nên đây là dạng quay lui ẩn (implicit backtracking).

Code mẫu (Python)

```python
class NQueensSolver:
    def __init__(self):
        self.solution_count = 0
```

```
    def solve_n_queens(self, n: int) -> int:
        self.solution_count = 0
        positions = [-1] * n  # positions[row] = col

        def is_safe(current_row: int, current_col: int) -> bool:
            for prev_row in range(current_row):
                prev_col = positions[prev_row]

                # Kiểm tra trùng cột
                if prev_col == current_col:
                    return False

                # Kiểm tra đường chéo (độ lệch hàng == độ lệch cột)
                if abs(current_row - prev_row) == abs(current_col - prev_col):
                    return False

            return True

        def place_queen(row: int) -> None:
            # 1. Điều kiện dừng: Đặt thành công N con hậu
            if row == n:
                self.solution_count += 1
                return

            # 2. Vòng lặp: Thử đặt hậu ở hàng 'row' vào từng cột 'col'
            for col in range(n):
                # 3. Kiểm tra tính hợp lệ
                if is_safe(row, col):
                    # 4. Thử (Try)
                    positions[row] = col

                    # 5. Đệ quy
                    place_queen(row + 1)

                    # 6. Quay lui (Implicit Backtracking):
                    # Không cần xóa thủ công vì 'positions[row]' sẽ bị ghi đè ở
vòng lặp sau.

        place_queen(0)
        return self.solution_count

# --- Chạy thử ---
if __name__ == "__main__":
    solver = NQueensSolver()
    n = 8
    print(f"Số cách đặt {n} con hậu là: {solver.solve_n_queens(n)}")
```

Vận dụng AI

- **Tối ưu hóa:** *"Code N-Queens bằng Python hiện tại kiểm tra đường chéo bằng vòng lặp. Làm sao dùng set() để theo dõi cột và đường chéo giúp hàm `is_safe` đạt độ phức tạp $O(1)$?"*
- **Debugging:** *"Tôi viết bài N-Queens cho N=4 mà kết quả ra 4 (đúng phải là 2). Đây là code Python của tôi [dán code]. Hãy tìm lỗi sai giúp tôi."*

Bài 4: [Trung bình/Khó] Giải Sudoku (Sudoku Solver)

Mô tả yêu cầu

Viết chương trình Python nhận vào một ma trận Sudoku 9x9 dạng `list[list[int]]`, trong đó các ô trống được biểu thị bằng số `0`.

Chương trình điền các số từ 1 đến 9 vào các ô trống để giải Sudoku và trả về `True` nếu giải thành công (đồng thời thay đổi ma trận ban đầu).

Gợi ý hướng dẫn

Hàm đệ quy `solve(board)`:

1. **Tìm ô trống:** Duyệt qua bảng để tìm ô trống đầu tiên `(r, c)` (ô có giá trị `0`).
2. **Điều kiện dừng:** Nếu không còn ô trống nào, bảng Sudoku đã được giải xong → Trả về `True`.
3. **Vòng lặp (Thử):** Tại ô trống `(r, c)`, thử điền từng số `num` từ `1` đến `9`.
4. **Kiểm tra hợp lệ (`is_valid`):** Kiểm tra xem số `num` có bị trùng trên hàng `r`, cột `c`, hoặc trong ô vuông 3×3 tương ứng hay không.
5. **Thử & Đệ quy:**
    - Nếu hợp lệ, gán `board[r][c] = num`.
    - Gọi đệ quy `solve(board)`. Nếu hàm đệ quy trả về `True` → Trả về `True`.
6. **Quay lui (Backtrack):**
    - Nếu nhánh đệ quy sau trả về `False` (dẫn tới ngõ cụt), hoàn tác: `board[r][c] = 0` và tiếp tục vòng lặp thử số tiếp theo.
    - Nếu đã thử từ 1-9 mà không số nào hợp lệ → Trả về `False`.

Code mẫu (Python)

```python
class SudokuSolver:
    def solve(self, board: list[list[int]]) -> bool:
```

```
        for r in range(9):
            for c in range(9):
                # 1. Tìm ô trống
                if board[r][c] == 0:
                    # 2. Vòng lặp: Thử các số từ 1 đến 9
                    for num in range(1, 10):
                        if self._is_valid(board, r, c, num):
                            # 3. Thử (Try)
                            board[r][c] = num

                            # 4. Đệ quy
                            if self.solve(board):
                                return True  # Tìm thấy lời giải

                            # 5. Quay lui (Backtrack)
                            board[r][c] = 0  # Hoàn tác

                    # Thử từ 1-9 đều thất bại ở ô này
                    return False

        # Điều kiện dừng: Không còn ô trống nào
        return True

    def _is_valid(self, board: list[list[int]], row: int, col: int, num: int) ->
bool:
        # Kiểm tra hàng và cột
        for i in range(9):
            if board[row][i] == num or board[i][col] == num:
                return False

        # Kiểm tra ô 3x3
        start_row, start_col = 3 * (row // 3), 3 * (col // 3)
        for r in range(3):
            for c in range(3):
                if board[start_row + r][start_col + c] == num:
                    return False

        return True

# --- Chạy thử ---
if __name__ == "__main__":
    sudoku_board = [
        [5, 3, 0, 0, 7, 0, 0, 0, 0],
        [6, 0, 0, 1, 9, 5, 0, 0, 0],
        [0, 9, 8, 0, 0, 0, 0, 6, 0],
        [8, 0, 0, 0, 6, 0, 0, 0, 3],
        [4, 0, 0, 8, 0, 3, 0, 0, 1],
        [7, 0, 0, 0, 2, 0, 0, 0, 6],
```

```
        [0, 6, 0, 0, 0, 0, 2, 8, 0],
        [0, 0, 0, 4, 1, 9, 0, 0, 5],
        [0, 0, 0, 0, 8, 0, 0, 7, 9]
    ]

    solver = SudokuSolver()
    if solver.solve(sudoku_board):
        print("Giải Sudoku thành công:")
        for row in sudoku_board:
            print(row)
```

Vận dụng AI

- **So sánh khái niệm:** *"Hãy giải thích sự khác biệt về cơ chế trả về (return value) giữa thuật toán giải Sudoku (dừng ngay khi tìm thấy 1 lời giải) và bài toán N-Queens (duyệt hết để đếm tất cả lời giải)."*

Bài 5: [Khó] Tổ Hợp Số (Combination Sum) - Có Tối Ưu Hóa

Mô tả yêu cầu

Cho một danh sách các số nguyên không trùng lặp `candidates` và một số nguyên `target`. Viết chương trình Python tìm tất cả các tổ hợp duy nhất (unique combinations) từ `candidates` sao cho tổng các phần tử bằng `target`.

- Cùng một số trong `candidates` có thể được chọn **nhiều lần**.
- Các tổ hợp `[2, 2, 3]` và `[2, 3, 2]` được coi là giống nhau.
- **Input:** `candidates = [2, 3, 6, 7]`, `target = 7`
- **Output:**

```
None
[[2, 2, 3], [7]]
```

Gợi ý hướng dẫn

Để tránh các tổ hợp trùng lặp thứ tự, mẹo là **chỉ cho phép hàm đệ quy chọn các số từ chỉ số `start` trở đi**.

Hàm đệ quy `find_combinations(remaining_target, start, current_combination)`:

1. **Điều kiện dừng (Thành công):** Nếu `remaining_target == 0`, thêm bản sao của `current_combination` vào kết quả (`results.append(list(current_combination))`) và `return`.
2. **Cắt tỉa (Pruning) & Vòng lặp:**
   - Sắp xếp danh sách `candidates` tăng dần trước.
   - Duyệt `i` từ `start` đến `len(candidates) - 1`.
   - Nếu `candidates[i] > remaining_target`, ngắt vòng lặp (`break`) vì các số sau đó chắc chắn cũng sẽ lớn hơn.
3. **Thử (Try):** Thêm `candidates[i]` vào `current_combination`.
4. **Đệ quy:** Gọi lại `find_combinations(remaining_target - candidates[i], i, current_combination)`.
   - **Lưu ý quan trọng:** Truyền `i` (không phải `i + 1`) vì bài toán cho phép tái sử dụng lại phần tử `candidates[i]`.
5. **Quay lui (Backtrack):** Xóa phần tử vừa thêm bằng `current_combination.pop()`.

Code mẫu (Python)

Python

```
class CombinationSumSolver:
    def combination_sum(self, candidates: list[int], target: int) ->
list[list[int]]:
        results = []
        candidates.sort()  # Sắp xếp mảng để dễ cắt tỉa nhánh đệ quy (pruning)

        def find_combinations(remaining_target: int, start: int,
current_combination: list[int]) -> None:
            # 1. Điều kiện dừng (Thành công)
            if remaining_target == 0:
                results.append(list(current_combination))  # Thêm một bản sao
(copy)
                return

            # 2. Vòng lặp các lựa chọn
            for i in range(start, len(candidates)):
                # Cắt tỉa nhánh (Pruning): nếu số hiện tại lớn hơn target còn lại,
dừng vòng lặp
                if candidates[i] > remaining_target:
                    break

                # 3. Thử (Try)
                current_combination.append(candidates[i])

                # 4. Đệ quy
```

```
                # Truyền 'i' (chứ không phải 'i + 1') để cho phép tái sử dụng số
candidates[i]
                find_combinations(remaining_target - candidates[i], i,
current_combination)

                # 5. Quay lui (Backtrack)
                current_combination.pop()

        find_combinations(target, 0, [])
        return results

# --- Chạy thử ---
if __name__ == "__main__":
    solver = CombinationSumSolver()
    candidates = [2, 3, 6, 7]
    target = 7
    print("Các tổ hợp tìm được:", solver.combination_sum(candidates, target))
```

Vận dụng AI

- **Giải thích thuật toán:** *"Giải thích tại sao việc truyền tham số `start` trong bài Combination Sum lại ngăn chặn được việc tạo ra các tổ hợp bị trùng lặp như `[2, 3, 2]` và `[2, 2, 3]`?"*
- **Mở rộng biến thể:** *"Nếu mỗi số trong candidates chỉ được dùng tối đa 1 lần (Bài toán Combination Sum II), tôi cần sửa đổi điều kiện đệ quy và truyền tham số `start` trong Python như thế nào?"*